TM-V71 Remote

User & Technical Manual

Kenwood TM-V71 web remote

Version 3.1

Kenwood TM-V71(A/E) web remote + WebRTC audio + HackRF panadapter, for a Raspberry Pi.

1 Overview

TM-V71 Remote is a modern, dependency-light web remote control for the Kenwood TM-V71(A/E) dual-band FM transceiver, built around a direct serial driver. It gives full radio control in the browser, two-way browser audio over WebRTC/Opus, complete memory-channel management, an optional HackRF panadapter, classic 5-tone selective calling, and a CW/RTTY digimodes decoder/encoder. It installs as a Progressive Web App (PWA) and is designed to run on a Raspberry Pi.

Unlike hamlib, whose TM-V71 backends are unreliable, this project speaks the radio's documented PC command set directly and exposes the radio's full feature set, including per-channel memory programming.

2 Features

- Full live control of both bands (A/B): frequency, VFO/memory mode, repeater shift & offset, CTCSS/DCS, step, control band, and PTT over CAT.
- Memory channels (CHIRP-level): read, write, delete, rename any of the 1000 channels, plus CSV import/export.
- Live status pushed to the browser over a WebSocket; transmit lights the UI.
- Two-way audio: direct WebRTC/Opus between browser and backend via aiortc; the mic feeds the radio only while PTT is engaged.
- Optional HackRF One waterfall: real-time panadapter (auto-following the tuned frequency) or a wideband sweep.
- Classic 5-tone selective calling (ZVEI-1/2, CCIR, EEA): call, decode, and mute RX until your own ID is received.
- CW (Morse) and RTTY (Baudot/AFSK) decode + encode, over the FM audio path.
- Installable PWA with a mobile landscape swipe-deck layout.
- Resilient operation: the phone screen is kept awake, browser audio auto-reconnects after a network glitch, and a backend watchdog releases a latched PTT if every client disappears.
- GPIO power switch, auto power-off, TX power, squelch, in-display S-meter.
- Two themes (dark/light); no build step for the UI.

3 Architecture

```
                      Raspberry Pi
/dev/ttyUSB0 (57600) -- tmv71 driver --+
                                       v
 FastAPI backend -- REST + WebSocket (live status)
   - control (freq/mode/band/PTT)  - memory CRUD + CSV
   - CW/RTTY + 5-tone selcall      - HackRF spectrum

 USB sound -- aiortc WebRTC <-> browser (Opus, PTT)
 FastAPI serves the SPA / PWA at "/" (HTTPS/TLS)
       ^ LAN (HTTPS)
  Browser  -  installable PWA (control + audio)
```

The backend owns the serial port directly (backend/app/tmv71.py). One FastAPI process serves the SPA/PWA, the REST control endpoints, the live-status WebSocket, and the WebRTC audio signalling — no extra services. Audio is 48 kHz / 16-bit / mono internally (Opus' native rate).

4 Requirements & Hardware

- Raspberry Pi (tested on Debian 13 / aarch64), Python 3.11+.
- Kenwood TM-V71(A/E) on a serial port (FTDI programming cable), 57600 baud.
- A USB sound interface wired to the radio (data port or mic/speaker), full-duplex.
- System packages: portaudio19-dev, swig + liblgpio-dev (optional GPIO).
- Optional: a HackRF One plus the hackrf host tools for the waterfall.

5 Installation

```
sudo apt-get install -y portaudio19-dev python3-venv swig liblgpio-dev
git clone https://github.com/CQ-DJ0SH/tmv71-remote.git
cd tmv71-remote/backend
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

For a reboot-proof setup, install the systemd unit from the deploy/ directory. The service runs uvicorn with TLS on port 8443.

6 Running over HTTPS

Browser microphone access (getUserMedia) and the PWA service worker require a secure context, so the server runs over HTTPS. A quick self-signed certificate is enough on the desktop (accept the warning once); for installing the PWA on a phone you need a trusted certificate (see chapter 9).

```
cd backend && mkdir -p certs
openssl req -x509 -newkey rsa:2048 -nodes -days 3650 \
  -keyout certs/key.pem -out certs/cert.pem -subj "/CN=tmv71-remote" \
  -addext "subjectAltName=IP:<pi-ip>,DNS:localhost,IP:127.0.0.1"
.venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8443 \
  --ssl-keyfile certs/key.pem --ssl-certfile certs/cert.pem
```

Open https://<pi-ip>:8443/ and accept the certificate once.

7 The Web Interface

Band panels (VFO A / VFO B)

Each band shows the frequency on a 7-segment display with an S-meter (1 s peak-hold), plus controls for VFO/memory mode, CTRL/PTT band selection, TX power, squelch (remembered per band across power cycles), repeater shift/offset, tone and bandwidth. The digit tuner lets you click bars above/below each digit to step the frequency; AIR Band tunes band A to the 118–137 MHz air band (receive-only).

PTT & memory quick keys

Hold the large PTT button (or the space bar) to transmit; PTT-LOCK latches transmit. PTT and PTT-LOCK require connected audio (there is no mic otherwise) — they are disabled while audio is off and released automatically if audio disconnects. ROGER adds a two-tone (1000/1750 Hz) beep on release; while transmitting the button shows a count-up timer (MM:SS). The 1750 Hz button arms a tone-call. The left column recalls memory channels 0–9 (the loaded channel's key glows); the right column sends DTMF memories. On mobile, mini RX/TX VU bars with peak-hold flank the button.

Audio (WebRTC/Opus)

Open the AUDIO panel, pick the audio band, click CONNECT and allow the microphone. RX and mic levels are shown live. Controls: RX/TX gain, a MIC TEST switch (meter the mic without keying — it records while on and replays your audio over RX when switched off, so you can hear how you sound, with no RF; the radio's RX is muted during the test), switchable TX and RX voice low-pass filters (≤ 3.5 kHz), a two-tone test, and TX timing (buffer / trail). The USB card mixer is in Settings > Audio. The audio link auto-reconnects after a network glitch and is restored automatically on the next launch.

Bluetooth headsets: transmit audio is captured from the phone's built-in microphone (not the headset's), so the headset stays on the A2DP profile and receive audio keeps coming through in good quality. Using the headset mic would force Android onto the mono HFP/SCO profile and, on many phones, leave RX stuck until Bluetooth is toggled.

HackRF waterfall

If a HackRF One is connected, this panel shows a live spectrum stacked over a waterfall: a panadapter centred on the tuned frequency (auto-following) or a wideband sweep. Receive-only; LNA/VGA gains and a display level are adjustable.

Selcall (classic 5-tone)

Send and decode classic selective calls (ZVEI-1/2, CCIR, EEA). Enter a 5-digit CALL code and press CALL (keys PTT). Enter your own ID and press MUTE to silence RX until your ID is received — then it un-mutes automatically. Over FM this is AFSK; use a dummy load when setting up.

Digimodes (CW / RTTY)

Switch between CW (Morse) and RTTY (Baudot/AFSK). DECODE shows received text; type into the field and SEND to transmit (keys PTT). Parameters: CW WPM/pitch — with an optional AUTO mode that tracks the received speed — and RTTY baud/shift/mark. Over the FM radio this is MCW / AFSK — not native HF CW/RTTY.

Band scan

Sweep a VHF/UHF range or the memory bank and see an occupancy spectrum + waterfall. Double-click a channel to tune the control VFO to it.

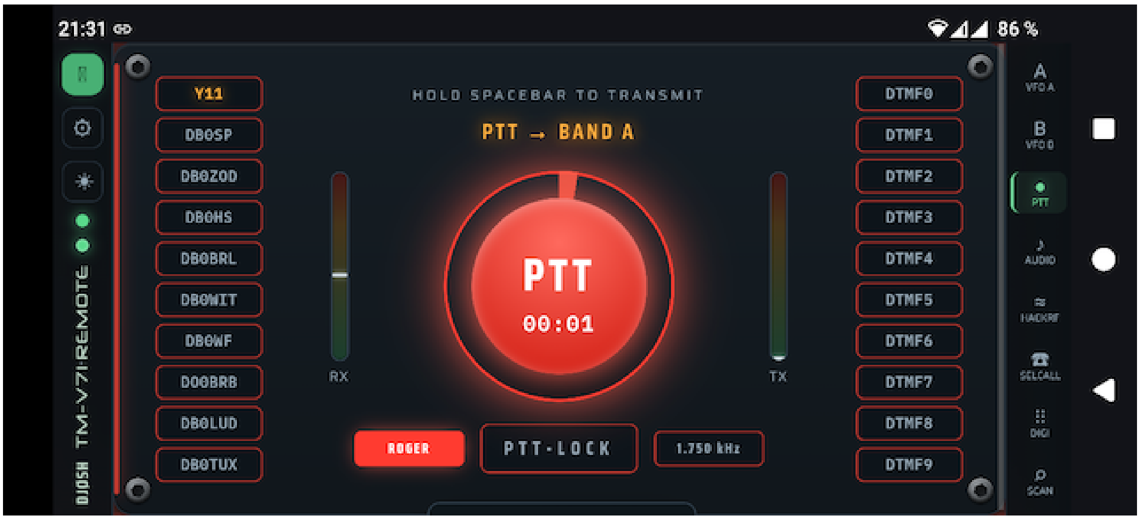
Settings

Tabs: General (callsign, API backend URL, serial port/baud, GPIO power, auto power-off, logo, GitHub self-update, Root-CA download), Audio (device, USB mixer, voice filters, test tone, TX timing), Rig-Info, Rig-Memory, Rig-DTMF, and Pi-Hardware (host metrics).

8 Mobile App (PWA)

The UI installs as a Progressive Web App: full-screen, with an app-shell service worker for instant launch. On phones the panels become a vertical swipe deck — swipe up/down, one panel per screen (this keeps the deck's scroll axis off the horizontal sliders, so they stay usable) — with the title bar as a slim vertical strip on the left and an icon tab rail on the right. Past the last panel is an info page listing the app version and browser/environment details. The app is forced to landscape; portrait shows a rotate hint. Install via the browser menu (Install / Add to Home Screen); on iOS use Safari > Share.

While the app is open the phone screen is kept awake via the Screen Wake Lock API, so it won't dim or lock mid-QSO. The browser audio link reconnects automatically after a brief network interruption, and if the connection to the backend is lost while transmit is latched, the PTT is released (locally and by a backend watchdog) so the rig can never stay keyed unattended.



9 Trusted Certificate (Root CA)

A self-signed certificate is fine on the desktop, but mobile browsers will not install the PWA or run the service worker without a trusted certificate. Create your own root CA, sign the server certificate with it, and trust the CA on the phone. A download link for the CA appears in Settings > General once it exists.

```
cd backend/certs && mkdir -p ca
# 1) Root CA (10 years) — keep ca.key secret
openssl genrsa -out ca/ca.key 4096
openssl req -x509 -new -key ca/ca.key -sha256 -days 3650 \
  -subj "/CN=TM-V71 Remote Root CA" \
  -addext "basicConstraints=critical,CA:TRUE,pathlen:0" \
  -addext "keyUsage=critical,keyCertSign,cRLSign" -out ca/ca.crt
# 2) server cert signed by the CA (list every name/IP in the SAN)
openssl genrsa -out key.pem 2048
openssl req -new -key key.pem -subj "/CN=tm-v71.example.lan" -out s.csr
openssl x509 -req -in s.csr -CA ca/ca.crt -CAkey ca/ca.key \
  -CAcreateserial -days 825 -sha256 -extfile leaf.ext -out cert.pem
sudo systemctl restart tmv71-remote.service
```

Install ca/ca.crt on the phone (Settings > Security > Install a certificate > CA certificate). The leaf is valid 825 days; re-issue it from the same CA without re-installing on phones. Keep ca/ca.key secret.

10 Configuration

Settings are read from environment variables (prefix TMV71_) or a .env file, with web-UI changes persisted to backend/app/runtime.json. Key variables:

```
TMV71_SERIAL_PORT=/dev/ttyUSB0   TMV71_SERIAL_BAUD=57600
TMV71_HOST=0.0.0.0               TMV71_PORT=8443
TMV71_AUDIO_DEVICE=NAD           TMV71_AUDIO_ENABLED=true
TMV71_GPIO_POWER_PIN=17          TMV71_CALLSIGN=DJ0SH
TMV71_SSL_CERTFILE=...  TMV71_SSL_KEYFILE=...
```

11 REST & WebSocket API

Control & status

```
GET  /api/status           live radio state
POST /api/frequency        set VFO frequency
POST /api/band-mode        VFO / memory / call
POST /api/control-band     select control band
POST /api/ptt              key / un-key (CAT)
POST /api/ptt-band         select TX band
POST /api/squelch /step    squelch level / step
POST /api/vfo              shift/offset/tone/bw
GET  /api/info /version    rig + app info
WS   /ws                   live status stream
```

Memory channels

```
GET    /api/memories?start&end   list channels
GET    /api/memories/{ch}        one channel
PUT    /api/memories/{ch}        write channel
DELETE /api/memories/{ch}        clear channel
GET    /api/memories.csv         export CSV
POST   /api/memories/import      import CSV
POST   /api/recall               recall to band
```

Audio

```
POST /api/webrtc/offer      WebRTC SDP offer
GET  /api/audio/status      RX/TX levels, flags
GET  /api/audio/devices     list sound devices
POST /api/audio/device      pick device
POST /api/audio/gain        rx_gain / tx_gain
POST /api/audio/buffer      tx buffer / ptt tail
POST /api/audio/tones       roger/test/lowpass/mic-test
GET/POST /api/audio/mixer   USB card mixer
```

Digimodes & selcall

```
GET  /api/digi              CW/RTTY status
POST /api/digi/config       mode + parameters
POST /api/digi/tx           encode + transmit
WS   /ws/digi               decoded text stream
GET  /api/selcall           5-tone status
POST /api/selcall/config    standard / tone / own
POST /api/selcall/tx        send a 5-tone call
WS   /ws/selcall            decoded calls stream
```

SDR & scan

```
GET  /api/hackrf            SDR status
POST /api/hackrf/start|stop|config
WS   /ws/hackrf             spectrum/waterfall frames
GET  /api/scan  POST /api/scan/start|stop  band scan
```

System & power

```
GET/POST /api/power-switch   GPIO power
POST /api/gpio-config        set GPIO pin
POST /api/auto-power-off     idle auto-off
GET/POST /api/serial-config  serial port/baud
GET/POST /api/callsign /theme
GET  /api/system             Pi host metrics
GET/POST /api/update         GitHub self-update
```

12 Troubleshooting

- No CAT / 'radio offline': check the serial port and baud in Settings; the FTDI cable must be on /dev/ttyUSB0 (or set the right port).
- No audio: ensure HTTPS, click CONNECT and allow the mic; check the USB card and its mixer levels (playback drives the radio mic on TX).
- PWA won't install / theme not switching on a phone: the certificate is not trusted — install the Root CA (chapter 9).
- RX filter only on one channel: reconnect audio (Disconnect/Connect) to renegotiate Opus to mono.
- Decoders need a clean signal; tune levels and (for RTTY) the mark tone.

13 Security

LAN-only by design; there is no authentication. Do not expose the port directly to the internet — use a VPN (WireGuard/Tailscale) or a reverse proxy with TLS + auth. The Root CA private key never leaves the Pi.

14 Credits & License

Kenwood PC protocol docs: LA3QMA/TM-V71_TM-D710-Kenwood. Built with aiortc (WebRTC/Opus) and sounddevice. Fonts: Saira, IBM Plex Mono, DSEG (7-segment), Neuropol (title). See the repository for license details.